

# Cloud Computing

**V Sahiti Yellanki**

**Department of CSE- AIML & IoT**

**VNR VJIET**

**UNIT-V:**

**Cloud Platforms in Industry:** Amazon web services: Compute services, Storage services, communication services, Additional services

**Google App Engine:** Architecture and core concepts, Application life cycle, Cost model, Observations

**Microsoft Azure:** Azure core concepts, SQL Azure, Windows Azure platform appliance

**Cloud Applications:** Scientific Applications: Healthcare: ECG analysis in the cloud

**Business and Consumer Applications:** CRM and ERP, Productivity, Social networking, Media applications

## Amazon web services:

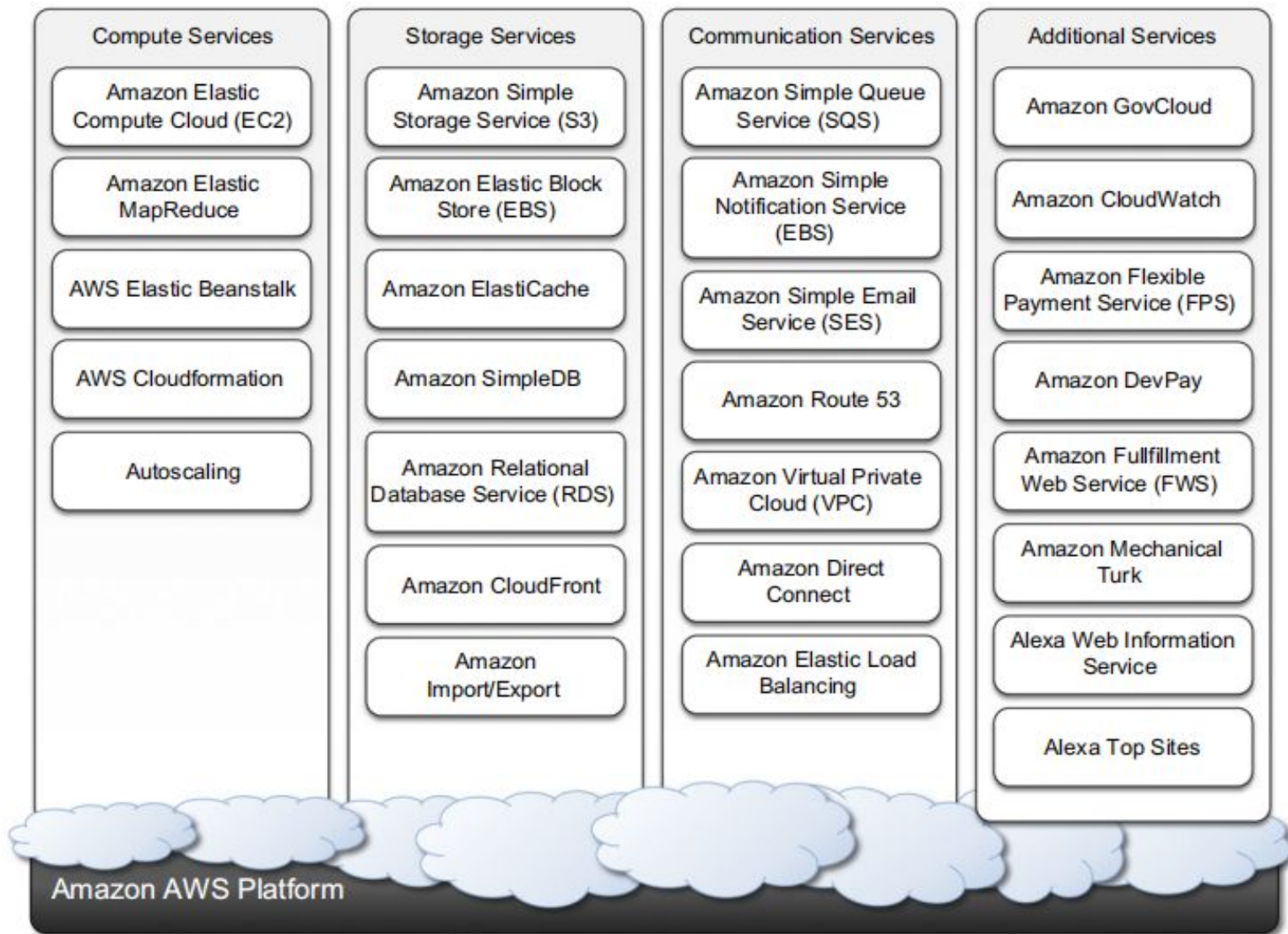
Amazon Web Services (AWS) is a platform that allows the development of flexible applications by providing solutions for elastic infrastructure scalability, messaging, and data storage. The platform is accessible through SOAP or RESTful Web service interfaces and provides a Web-based console where users can handle administration and monitoring of the resources required, as well as their expenses computed on a **pay-as-you-go basis**.

Diagram shows all the services available in the AWS ecosystem. At the base of the solution stack are services that provide raw compute and raw storage: Amazon Elastic Compute (EC2) and Amazon Simple Storage Service (S3).

**At the higher level**, Elastic MapReduce and AutoScaling provide additional capabilities for building smarter and more elastic computing systems. On the **data side**, Elastic Block Store (EBS), Amazon SimpleDB, Amazon RDS, and Amazon ElastiCache provide solutions for reliable data snapshots and the management of structured and semistructured data. Communication needs are covered at the **networking level** by Amazon Virtual Private Cloud (VPC), Elastic Load Balancing, Amazon Route 53, and Amazon Direct Connect. More advanced services for connecting applications are Amazon Simple Queue Service (SQS), Amazon Simple Notification Service (SNS), and Amazon Simple E-mail Service (SES). Other services include:

- Amazon CloudFront content delivery network solution
- Amazon CloudWatch monitoring solution for several Amazon services
- Amazon Elastic BeanStalk and CloudFormation flexible application packaging and deployment

As shown, AWS comprise a wide set of services.



**FIGURE 9.1**

Amazon Web Services ecosystem.



**1. Compute services:** Compute services constitute the fundamental element of cloud computing systems. The fundamental service in this space is Amazon EC2, which delivers an IaaS solution that has served as a reference model for several offerings from other vendors in the same market segment. Amazon EC2 allows deploying servers in the form of virtual machines created as instances of a specific image. Images come with a preinstalled operating system and a software stack, and instances can be configured for memory, number of processors, and storage. Users are provided with credentials to remotely access the instance and further configure or install software if needed.

### **1.1 Amazon machine images:**

Amazon Machine Images (AMIs) are templates from which it is possible to create a virtual machine. They are stored in Amazon S3 and identified by a unique identifier in the form of ami-xxxxxx and a manifest XML file.

An AMI contains a physical file system layout with a predefined operating system installed. These are specified by the Amazon Ramdisk Image (ARI, id: ari-yyyyyy) and the Amazon Kernel Image (AKI, id: aki-zzzzzz), which are part of the configuration of the template.

AMIs are either created from scratch or “bundled” from existing EC2 instances. A common practice is to prepare new AMIs to create an instance from a preexisting AMI, log into it once it is booted and running, and install all the software needed. Using the tools provided by Amazon, we can convert the instance into a new image.

Once an AMI is created, it is stored in an S3 bucket and the user can decide whether to make it available to other users or keep it for personal use. Finally, it is also possible to associate a product code with a given AMI, thus allowing the owner of the AMI to get revenue every time this AMI is used to create EC2 instances.

**1.2 .EC2 instances:** EC2 instances represent virtual machines. They are created using AMI as templates, which are specialized by selecting the number of cores, their computing power, and the installed memory.

The processing power is expressed in terms of virtual cores and EC2 Compute Units (ECUs).

The ECU is a measure of the computing power of a virtual core; it is used to express a predictable quantity of real CPU power that is allocated to an instance. By using compute units instead of real frequency values, Amazon can change over time the mapping of such units to the underlying real amount of computing power allocated, thus keeping the performance of EC2 instances consistent with standards set by the times.

Over time, the hardware supporting the underlying infrastructure will be replaced by more powerful hardware, and the use of ECUs helps give users a consistent view of the performance offered by EC2 instances. Since users rent computing capacity rather than buying hardware, this approach is reasonable.

Table shows all the currently available configurations for EC2 instances. We can identify six major categories:

- **Standard instances.** This class offers a set of configurations that are suitable for most applications. EC2 provides three different categories of increasing computing power, storage, and memory.
- **Micro instances.** This class is suitable for those applications that consume a limited amount of computing power and memory and occasionally need bursts in CPU cycles to process surges in the workload. Micro instances can be used for small Web applications with limited traffic.
- **High-memory instances.** This class targets applications that need to process huge workloads and require large amounts of memory. Three-tier Web applications characterized by high traffic are the target profile. Three categories of increasing memory and CPU are available, with memory proportionally larger than computing power.



- **High-CPU instances.** This class targets compute-intensive applications. Two configurations are available where computing power proportionally increases more than memory.
- **Cluster Compute instances.** This class is used to provide virtual cluster services. Instances in this category are characterized by high CPU compute power and large memory and an extremely high I/O and network performance, which makes it suitable for HPC applications.
- **Cluster GPU instances.** This class provides instances featuring graphic processing units (GPUs) and high compute power, large memory, and extremely high I/O and network performance. This class is particularly suited for cluster applications that perform heavy graphic computations, such as rendering clusters.

**Table 9.2** Amazon EC2 (On-Demand) Instances Characteristics

| Instance Type         | ECU           | Platform  | Memory  | Disk Storage | Price (U.S. East)<br>(USD/hour) |
|-----------------------|---------------|-----------|---------|--------------|---------------------------------|
| Standard instances    |               |           |         |              |                                 |
| Small                 | 1(1 × 1)      | 32 bit    | 1.7 GB  | 160 GB       | \$0.085 Linux \$0.12 Windows    |
| Large                 | 4(2 × 2)      | 64 bit    | 7.5 GB  | 850 GB       | \$0.340 Linux \$0.48 Windows    |
| Extra Large           | 8(4 × 2)      | 64 bit    | 15 GB   | 1,690 GB     | \$0.680 Linux \$0.96 Windows    |
| Micro instances       |               |           |         |              |                                 |
| Micro                 | < = 2         | 32/64 bit | 613 MB  | EBS Only     | \$0.020 Linux \$0.03 Windows    |
| High-Memory instances |               |           |         |              |                                 |
| Extra Large           | 6.5(2 × 3.25) | 64 bit    | 17.1 GB | 420 GB       | \$0.500 Linux \$0.62 Windows    |
| Double Extra Large    | 13(4 × 3.25)  | 64 bit    | 34.2 GB | 850 GB       | \$1.000 Linux \$1.24 Windows    |
| Quadruple Extra Large | 26(8 × 3.25)  | 64 bit    | 68.4 GB | 1,690 GB     | \$2.000 Linux \$2.48 Windows    |
| High-CPU instances    |               |           |         |              |                                 |
| Medium                | 5(2 × 2.5)    | 32 bit    | 1.7 GB  | 350 GB       | \$0.170 Linux \$0.29 Windows    |
| Extra Large           | 20(8 × 2.5)   | 64 bit    | 7 GB    | 1,690 GB     | \$0.680 Linux \$1.16 Windows    |
| Cluster instances     |               |           |         |              |                                 |
| Quadruple Extra Large | 33.5          | 64 bit    | 23 GB   | 1,690 GB     | \$1.600 Linux \$1.98 Windows    |
| Cluster GPU instances |               |           |         |              |                                 |
| Quadruple Extra Large | 33.5          | 64 bit    | 22 GB   | 1,690 GB     | \$2.100 Linux \$2.60 Windows    |

- EC2 instances can be run either by using the command-line tools provided by Amazon, which connects the Amazon Web Service that provides remote access to the EC2 infrastructure, or via the AWS console, which allows the management of other services, such as S3. By default an EC2 instance is created with the kernel and the disk associated to the AMI.

### **1.3 EC2 environment:**

EC2 instances are executed within a virtual environment, which provides them with the services they require to host applications. The EC2 environment is in charge of allocating addresses, attaching storage volumes, and configuring security in terms of access control and network connectivity.

By default, instances are created with an internal IP address, which makes them capable of communicating within the EC2 network and accessing the Internet as clients. It is possible to associate an Elastic IP to each instance, which can then be remapped to a different instance over time .

Elastic IPs allow instances running in EC2 to act as servers reachable from the Internet and, since they are not strictly bound to specific instances, to implement failover capabilities. Together with an external IP, EC2 instances are also given a domain name that generally is in the form `ec2-xxx_x0002_xxx-xxx.compute-x.amazonaws.com`, where `xxx-xxx-xxx` normally represents the four parts of the external IP address separated by a dash, and `compute-x` gives information about the availability zone where instances are deployed.

### **1.4 Advanced compute services:**

EC2 instances and AMIs constitute the basic blocks for building an IaaS computing cloud.

#### **AWS CloudFormation:**

AWS CloudFormation constitutes an extension of the simple deployment model that characterizes EC2 instances. CloudFormation introduces the concepts of templates, which are JSON formatted text files that describe the resources needed to run an application or a service in EC2 together with the relations between them.

CloudFormation allows easily and explicitly linking EC2 instances together and introducing dependencies among them. Templates provide a simple and declarative way to build complex systems and integrate EC2 instances with other AWS services such as S3, SimpleDB, SQS, SNS, Route 53, Elastic Beanstalk, and others.



### AWS elastic beanstalk:

- AWS Elastic Beanstalk constitutes a simple and easy way to package applications and deploy them on the AWS Cloud. This service simplifies the process of provisioning instances and deploying application code and provides appropriate access to them. Currently, this service is available only for Web applications developed with the Java/Tomcat technology stack. Developers can conveniently package their Web application into a WAR file and use Beanstalk to automate its deployment on the AWS Cloud.

### Amazon elastic MapReduce:

Amazon Elastic MapReduce provides AWS users with a cloud computing platform for MapReduce applications. It utilizes Hadoop as the MapReduce engine, deployed on a virtual infrastructure composed of EC2 instances, and uses Amazon S3 for storage needs.

Apart from supporting all the application stack connected to Hadoop (Pig, Hive, etc.), Elastic MapReduce introduces elasticity and allows users to dynamically size the Hadoop cluster according to their needs, as well as select the appropriate configuration of EC2 instances to compose the cluster (Small, High-Memory, High-CPU, Cluster Compute, and Cluster GPU).

## 2. Storage services

AWS provides a collection of services for data storage and information management. The core service in this area is represented by Amazon Simple Storage Service (S3). This is a distributed object store that allows users to store information in different formats. The core components of S3 are two: buckets and objects. Buckets represent virtual containers in which to store objects; objects represent the content that is actually stored. Objects can also be enriched with metadata that can be used to tag the stored content with additional information.

### 2.1. S3 key concepts

As the name suggests, S3 has been designed to provide a simple storage service that's accessible through a Representational State Transfer (REST) interface, which is quite similar to a distributed file system but which presents some important differences that allow the infrastructure to be highly efficient:

- The storage is organized in a two-level hierarchy. S3 organizes its storage space into buckets that cannot be further partitioned. This means that it is not possible to create directories or other kinds of physical groupings for objects stored in a bucket.
- Stored objects cannot be manipulated like standard files. S3 has been designed to essentially provide storage for objects that will not change over time. Therefore, it does not allow renaming, modifying, or relocating an object. Once an object has been added to a bucket, its content and position is immutable, and the only way to change it is to remove the object from the store and add it again.
- Content is not immediately available to users. The main design goal of S3 is to provide an eventually consistent data store. As a result, because it is a large distributed storage facility, changes are not immediately reflected.
- Requests will occasionally fail. Due to the large distributed infrastructure being managed, requests for object may occasionally fail. Under certain conditions, S3 can decide to drop a request by returning an internal server error. Therefore, it is expected to have a small failure rate during day-to-day operations, which is generally not identified as a persistent failure.

Access to S3 is provided with RESTful Web services. These express all the operations that can be performed on the storage in the form of HTTP requests (GET, PUT, DELETE, HEAD, and POST), which operate differently according to the element they address. As a rule of thumb PUT/ POST requests add new content to the store, GET/HEAD requests are used to retrieve content and information, and DELETE requests are used to remove elements or information attached to them.

## Resource naming

Buckets, objects, and attached metadata are made accessible through a REST interface. Therefore, they are represented by uniform resource identifiers (URIs) under the `s3.amazonaws.com` domain. All the operations are then performed by expressing the entity they are directed to in the form of a request for a URI. Amazon offers three different ways of addressing a bucket:

Canonical form: `http://s3.amazonaws.com/bucket_name/`. The bucket name is expressed as a path component of the domain name `s3.amazonaws.com`

Subdomain form: `http://bucketname.s3.amazonaws.com/`. Alternatively, it is also possible to reference a bucket as a subdomain of `s3.amazonaws.com`. To express a bucket name in this

form, the name has to do all of the following:

- Be between 3 and 63 characters long
- Contain only letters, numbers, periods, and dashes
- Start with a letter or a number
- Contain at least one letter
- Have no fragments between periods that start with a dash or end with a dash or that are empty strings

Virtual hosting form: `http://bucket-name.com/`. Amazon also allows referencing of its resources with custom URLs. This is accomplished by entering a CNAME record into the DNS that points to the subdomain form of the bucket URI.

Since S3 is logically organized as a flat data store, all the buckets are managed under the `s3.amazonaws.com` domain.

Objects are always referred as resources local to a given bucket. Therefore, they always appear as a part of the resource component of a URI. Since a bucket can be expressed in three different ways, objects indirectly inherit this flexibility:

- Canonical form: `http://s3.amazonaws.com/bucket_name/object_name`
- Subdomain form: `http://bucket-name/s3.amazonaws.com/object_name`
- Virtual hosting form: `http://bucket-name.com/object_name`

## Buckets

A bucket is a container of objects. It can be thought of as a virtual drive hosted on the S3 distributed storage, which provides users with a flat store to which they can add objects. Buckets are top-level elements of the S3 storage architecture and do not support nesting. That is, it is not possible to create “subbuckets” or other kinds of physical divisions.

A bucket is located in a specific geographic location and eventually replicated for fault tolerance and better content distribution. Users can select the location at which to create buckets, which by default are created in Amazon's U.S. datacenters. Once a bucket is created, all the objects that belong to the bucket will be stored in the same availability zone of the bucket.

## Objects and metadata

Objects constitute the content elements stored in S3. Users either store files or push to the S3 text stream representing the object's content. An object is identified by a name that needs to be unique within the bucket in which the content is stored. The name cannot be longer than 1,024 bytes when encoded in UTF-8, and it allows almost any character. Since buckets do not support nesting, even characters normally used as path separators are allowed. This actually compensates for the lack of a structured file system, since directories can be emulated by properly naming objects.

## Access control and security

Amazon S3 allows controlling the access to buckets and objects by means of Access Control Policies (ACPs). An ACP is a set of grant permissions that are attached to a resource expressed by means of an XML configuration file.

Currently, five different permissions can be used:

- READ allows the grantee to retrieve an object and its metadata and to list the content of a bucket as well as getting its metadata.
- WRITE allows the grantee to add an object to a bucket as well as modify and remove it.
- READ\_ACP allows the grantee to read the ACP of a resource.
- WRITE\_ACP allows the grantee to modify the ACP of a resource.
- FULL\_CONTROL grants all of the preceding permissions.

## 2.2 Amazon elastic block store

The Amazon Elastic Block Store (EBS) allows AWS users to provide EC2 instances with persistent storage in the form of volumes that can be mounted at instance startup. They accommodate up to TB of space and are accessed through a block device interface, thus allowing users to format them according to the needs of the instance they are connected to (raw storage, file system, or other). The content of an EBS volume survives the instance life cycle and is persisted into S3. EBS volumes can be cloned, used as boot partitions, and constitute durable storage since they rely on S3 and it is possible to take incremental snapshots of their content.

EBS volumes normally reside within the same availability zone of the EC2 instances that will use them to maximize the I/O performance. It is also possible to connect volumes located in different availability zones. Once mounted as volumes, their content is lazily loaded in the background and according to the request made by the operating system. This reduces the number of I/O requests that go to the network.

## 2.3 Amazon ElastiCache

ElastiCache is an implementation of an elastic in-memory cache based on a cluster of EC2 instances. It provides fast data access from other EC2 instances through a Memcached-compatible protocol so that existing applications based on such technology do not need to be modified and can transparently migrate to ElastiCache.

ElastiCache is based on a cluster of EC2 instances running the caching software, which is made available through Web services. An ElastiCache cluster can be dynamically resized according to the demand of the client applications.

## 2.4 Structured storage solutions

Enterprise applications quite often rely on databases to store data in a structured form, index, and perform analytics against it. Traditionally, RDBMS have been the common data back-end for a wide range of applications, even though recently more scalable and lightweight solutions have been proposed. Amazon provides applications with structured storage services in three different forms: preconfigured EC2 AMIs, Amazon Relational Data Storage (RDS), and Amazon SimpleDB.

### Preconfigured EC2 AMIs:

Preconfigured EC2 AMIs are predefined templates featuring an installation of a given database management system. EC2 instances created from these AMIs can be completed with an EBS volume for storage persistence. Available AMIs include installations of IBM DB2, Microsoft SQL Server, MySQL, Oracle, PostgreSQL, Sybase, and Vertica.

### Amazon RDS

RDS is relational database service that relies on the EC2 infrastructure and is managed by Amazon. Developers do not have to worry about configuring the storage for high availability, designing failover strategies, or keeping the servers up-to-date with patches.

Two relational engines are available: MySQL and Oracle. Two key advanced features of RDS are multi-AZ deployment and read replicas. The first option provides users with a failover infrastructure for their RDBMS solutions. The high-availability solution is implemented by keeping in standby synchronized copies of the services in different availability zones that are activated if the primary service goes down.

### Amazon SimpleDB

Amazon SimpleDB is a lightweight, highly scalable, and flexible data storage solution for applications that do not require a fully relational model for their data. SimpleDB provides support for semistructured data, the model for which is based on the concept of domains, items, and attributes. With respect to the relational model, this model provides fewer constraints on the structure of data entries, thus obtaining improved performance in querying large quantities of data. As happens for Amazon RDS, this service frees AWS users from performing configuration, management, and high-availability design for their data stores.

## 2.5 Amazon CloudFront

CloudFront is an implementation of a content delivery network on top of the Amazon distributed storage infrastructure. It leverages a collection of edge servers strategically located around the globe to better serve requests for static and streaming Web content so that the transfer time is reduced as much as possible.

AWS provides users with simple Web service APIs to manage CloudFront. To make available content through CloudFront, it is necessary to create a distribution. This identifies an origin server, which contains the original version of the content being distributed, and it is referenced by a DNS domain under the Cloudfront.net domain name (i.e., my-distribution.Cloudfront.net). It is also possible to map a given domain name to a distribution.



## 3 Communication services:

Amazon provides facilities to structure and facilitate the communication among existing applications and services residing within the AWS infrastructure. These facilities can be organized into two major categories: virtual networking and messaging.

### 3.1 Virtual networking

Virtual networking comprises a collection of services that allow AWS users to control the connectivity to and between compute and storage services. Amazon Virtual Private Cloud (VPC) and Amazon Direct Connect provide connectivity solutions in terms of infrastructure; Route 53 facilitates connectivity in terms of naming.

Amazon VPC provides a great degree of flexibility in creating virtual private networks within the Amazon infrastructure and beyond. The service providers prepare either templates covering most of the usual scenarios or a fully customizable network service for advanced configurations.

**Amazon Direct Connect** allows AWS users to create dedicated networks between the user private network and Amazon Direct Connect locations, called ports. This connection can be further partitioned in multiple logical connections and give access to the public resources hosted on the Amazon infrastructure. The advantage of using Direct Connect versus other solutions is the consistent performance of the connection between the users' premises and the Direct Connect locations. This service is compatible with other services such as EC2, S3, and Amazon VPC and can be used in scenarios requiring high bandwidth between the Amazon network and the outside world.

**Amazon Route 53** implements dynamic DNS services that allow AWS resources to be reached through domain names different from the amazon.com domain. By leveraging the large and globally distributed network of Amazon DNS servers, AWS users can expose EC2 instances or S3 buckets as resources under a domain of their property, for which Amazon DNS servers become authoritative.

## 3.2 Messaging

Messaging services constitute the next step in connecting applications by leveraging AWS capabilities. The three different types of messaging services offered are Amazon Simple Queue Service (SQS), Amazon Simple Notification Service (SNS), and Amazon Simple Email Service (SES).

Amazon SQS constitutes a disconnected model for exchanging messages between applications by means of message queues, hosted within the AWS infrastructure. Using the AWS console or directly the underlying Web service AWS, users can create an unlimited number of message queues and configure them to control their access.

Amazon SNS provides a publish-subscribe method for connecting heterogeneous applications. With respect to Amazon SQS, where it is necessary to continuously poll a given queue for a new message to process, Amazon SNS allows applications to be notified when new content of interest is available.

Amazon SES provides AWS users with a scalable email service that leverages the AWS infrastructure. Once users are signed up for the service, they have to provide an email that SES will use to send emails on their behalf. To activate the service, SES will send an email to verify the given address and provide the users with the necessary information for the activation.

## 3.4 Additional services

Besides compute, storage, and communication services, AWS provides a collection of services that allow users to utilize services in aggregation. The two relevant services are Amazon CloudWatch and Amazon Flexible Payment Service (FPS).

**Amazon CloudWatch** is a service that provides a comprehensive set of statistics that help developers understand and optimize the behavior of their application hosted on AWS. CloudWatch collects information from several other AWS services: EC2, S3, SimpleDB, CloudFront, and others. Using CloudWatch, developers can see a detailed breakdown of their usage of the service they are renting on AWS and can devise more efficient and cost-saving applications. Earlier services of CloudWatch were offered only through subscription, but now it is made available for free to all the AWS users.

Amazon FPS infrastructure allows AWS users to leverage Amazon's billing infrastructure to sell goods and services to other AWS users. Using Amazon FPS, developers do not have to set up alternative payment methods, and they can charge users via a billing service.